Hardware — Beginner Lab Skills (supplement to V1.1)

*Companion to hardware.md. Read this **before the first lab session**. It covers the practical skills a first-time builder needs before Part B makes sense: using a breadboard, a multimeter, resistor values, schematic symbols, uploading code, and digital inputs.*

Status: Drafted — needs technical review. Not yet tested in a lab session.

Section	Topic	Learners can…	Activity
L1	Breadboards	Place parts so they connect (or don't) as intended	Continuity check of the rows
L2	Multimeter	Measure voltage, resistance and continuity safely	Measure the LDR bright and dark
L3	Resistor values	Read colour bands and pick a standard value	Identify the 10 kΩ and 330 Ω resistors
L4	Arduino IDE	Upload a sketch and read the Serial Monitor	Upload sketch 01
L5	Schematic symbols	Read a simple schematic	Match symbols to the BOM
L6	Digital inputs	Explain pull-up resistors and debouncing	Sketch 05 (button)
L7	Glossary	Use the unit's terms consistently	—
L8	Practice questions	Check understanding	Questions H–L

L1. Breadboards: which holes are connected

A **breadboard** (solderless breadboard) lets you build a circuit by pushing component legs and wires into holes. Inside, metal strips connect some holes together.

```
+ ──────────────────────────────  (power rail: whole row connected)
- ──────────────────────────────  (ground rail: whole row connected)

   1   2   3   4   5  ...
a  o   o   o   o   o
b  o   o   o   o   o     each numbered COLUMN of a–e
c  o   o   o   o   o     is one connected strip of 5 holes
d  o   o   o   o   o
e  o   o   o   o   o
   ═══════ centre gap ═══════   (no connection across it)
f  o   o   o   o   o
g  o   o   o   o   o     f–j is a separate strip of 5
h  o   o   o   o   o
```

```
  i  o   o   o   o   o
  j  o   o   o   o   o

  + ──────────────────────────────
  – ──────────────────────────────
```

Rules:

- Holes **a–e in the same column** are connected. So are **f–j**. The two halves are **not** connected across the centre gap.
- The long **power rails** along the edges run the length of the board. Use red/+ for 5 V and blue/− for GND.
- **Many full-size breadboards split the rails halfway along.** If parts at one end get no power, add a short jumper across the break, or check with a multimeter's continuity mode (L2).
- A component with **both legs in the same strip is shorted out** — current goes through the strip, not the part. A resistor must bridge **two different** columns.
- Chips and push buttons sit **across the centre gap**, so each pin gets its own strip.

Common mistake: putting the LED's two legs in the same column. It will never light.

L2. Using a multimeter

A **multimeter** measures voltage, resistance, continuity and (carefully) current. It is the main tool for finding faults.

Measurement	Dial setting	How to connect	Power on?
DC voltage	V⎓ (or DCV), range 20 V	**Across** the part (in parallel): black on GND, red on the point to test	Yes
Resistance	Ω	Across the part. Take it out of the circuit, or lift one leg	**No** — power off
Continuity	🔊 / diode symbol	Across the two points. A beep means they are connected	**No** — power off
Current	A or mA, red lead in the **mA socket**	**In series**: break the circuit and let the current flow through the meter	Yes

Safety and accuracy:

- **Voltage is measured across, current is measured in series.** If the meter is set to current and you put it across a supply, you have made a short circuit. This can blow the meter's fuse or damage the board.
- **After measuring current, move the red lead back to the V/Ω socket.** Forgetting this is the most common way meters get damaged.
- **Measure resistance with the power off.** A powered circuit gives wrong readings and can damage the meter.
- Start on a higher range if your meter is not auto-ranging.

Lab activity — measure your LDR: with the LDR out of the circuit, measure its resistance (1) under room light, (2) covered by your hand, (3) under a phone torch. Record the three values in the project log. This replaces the "illustrative" values in B6 with real ones for *your* part.

Lab activity — check the divider: with the circuit built and powered, measure the voltage between A0 and GND. Calculate the ADC code you expect ($V/5 \times 1024$) and compare it with the Serial Monitor reading.

L3. Reading resistor values

Most through-hole resistors show their value as **colour bands**. On a 4-band resistor, the first two bands are digits, the third is a multiplier, and the fourth (usually gold or silver, a little apart) is the tolerance.

Colour	Digit	Multiplier
Black	0	×1
Brown	1	×10
Red	2	×100
Orange	3	×1 000
Yellow	4	×10 000
Green	5	×100 000
Blue	6	×1 000 000
Violet	7	—
Grey	8	—
White	9	—
Gold	—	×0.1 (as 4th band: **±5 %** tolerance)
Silver	—	×0.01 (as 4th band: **±10 %** tolerance)

Examples from the BOM:

- **10 kΩ** = brown (1), black (0), orange (×1000), gold (±5 %) → $10 \times 1000 = 10\,000\ \Omega$
- **330 Ω** = orange (3), orange (3), brown (×10), gold (±5 %) → $33 \times 10 = 330\ \Omega$

Tolerance is how far the real value may be from the printed value. A 10 kΩ ±5 % resistor can be anywhere from 9.5 kΩ to 10.5 kΩ. This is one reason real ADC readings differ from the worked examples (B7).

Standard values (E12 series): resistors are sold in fixed steps, not every value. Each decade uses these numbers:

> 10, 12, 15, 18, 22, 27, 33, 39, 47, 56, 68, 82

…then 100, 120, 150 … 1 k, 1.2 k, 1.5 k … and so on. That is why B8 calculates 300 Ω but uses **330 Ω**: it is the next standard value above, so the current stays below the target.

Accessibility note: colour bands are hard to read for colour-blind learners and in poor light. Always confirm the value with a multimeter (L2) — that is good practice for everyone.

L4. Uploading code with the Arduino IDE

1. **Install** the Arduino IDE (version 2.x) from https://www.arduino.cc/en/software.
2. **Connect** the Uno R3 with a USB cable. The green ON light should come on.
3. **Open** a sketch, e.g. `arduino/01_light_indicator_basic`.
4. **Choose the board:** Tools → Board → Arduino AVR Boards → **Arduino Uno**.
5. **Choose the port:** Tools → Port. On Windows it looks like `COM3`; on macOS like `/dev/cu.usbmodem…`. If you are unsure, unplug the board and see which port disappears.
6. **Verify** (✓) to check the code compiles, then **Upload** (→). The TX/RX lights flash, then the IDE says "Done uploading".
7. **Open the Serial Monitor** (magnifying-glass icon, top right) and set it to **9600 baud** — the same number as `Serial.begin(9600)`. The **Serial Plotter** draws the same numbers as a live graph.

Problem	Likely cause	Fix
No port appears	Charge-only USB cable, or driver missing	Try a different cable; for clone boards with a CH340 chip, install the CH340 driver
"Port busy" / "Access denied"	Another program (or a second Serial Monitor) has the port open	Close the other program and try again
`avrdude: stk500_recv(): programmer is not responding`	Wrong board or port, or something connected to pins 0/1	Re-check Tools → Board and Port; disconnect wires on pins 0 and 1 (they are used for USB serial)
Serial Monitor shows random symbols	Baud rate mismatch	Set the monitor to 9600 to match `Serial.begin(9600)`
`'xyz' was not declared in this scope`	Typo, or wrong capital letters (C++ is case-sensitive)	Check spelling: `digitalWrite`, not `digitalwrite`

L5. Schematic symbols

A **schematic** is a diagram that shows *how parts are connected*, not where they physically sit. The ASCII drawings in hardware.md are simplified schematics. Standard symbols:

Part	Symbol (described)	Notes
Resistor	Zig-zag line (US) or small rectangle (IEC/UK/EU)	No polarity
LDR	Resistor symbol with two arrows pointing **in** (light arriving)	No polarity
LED	Triangle pointing at a bar, with two arrows pointing **out**	Triangle side = anode (+), bar = cathode (−)
Diode	Triangle pointing at a bar	Current flows in the direction of the triangle

Part	Symbol (described)	Notes
NPN transistor	Circle with three leads; arrow on the emitter pointing **out**	Leads: collector, base, emitter
Push button	Two contacts with a lifted bar above them	No polarity
Ground (GND)	Three horizontal lines getting shorter, or a downward triangle	0 V reference
Supply (VCC / 5V)	Upward arrow or short bar labelled with the voltage	—
Capacitor	Two parallel lines (one curved or marked + if polarised)	Electrolytic capacitors have polarity

Activity: draw the light-indicator circuit (B6 + B8) using these symbols, then compare it with a classmate's. Two correct schematics can look different on the page and still show the same connections.

Free tools for drawing schematics: KiCad (full electronics design), Fritzing (breadboard view), or Wokwi (online Arduino simulator — useful for testing the sketches before the lab).

L6. Digital inputs: buttons, pull-up resistors and debouncing

Part B only uses an **analog** input (the LDR on A0). Many sensors and switches are **digital**: the pin reads either HIGH or LOW. The simplest example is a push button.

Floating inputs

If a pin is connected to a button and nothing else, then while the button is released the pin is connected to **nothing**. It is **floating**: it picks up electrical noise and reads random HIGH and LOW values. Touching the wire can change the reading.

Pull-up and pull-down resistors

A **pull-up resistor** (often 10 kΩ) connects the pin weakly to 5 V, so it reads **HIGH** when the button is released. The button connects the pin to GND, so pressing it reads **LOW**. A **pull-down resistor** does the opposite: connects weakly to GND, and the button connects to 5 V.

```
    5V                          Pin 2 ──[button]── GND
     │
  [10 kΩ]   pull-up             With INPUT_PULLUP, the 10 kΩ
     │                          is inside the chip, so only
Pin 2 ──[button]── GND          the button is needed.
```

The ATmega328P has **built-in pull-up resistors** (about 20–50 kΩ (secondary source)). Turn one on with:

```
pinMode(2, INPUT_PULLUP);   // released = HIGH, pressed = LOW
```

Beginners often expect "pressed = HIGH". With a pull-up it is the other way round — point this out, as with the LDR direction in B6.

Debouncing

The metal contacts inside a button **bounce** for a few milliseconds when pressed, so the pin can flip HIGH/LOW several times in one press. A program that counts presses will count too many. **Debouncing** means ignoring changes until the signal has been steady for about 20–50 ms. Sketch 05_button_override shows how, using `millis()` instead of `delay()` so the program never stops reading.

Analog vs digital sensors

	Analog sensor	**Digital sensor**
Output	A voltage anywhere in a range	HIGH/LOW, or data sent over a bus
Uno input	A0–A5 with `analogRead()`	Any digital pin with `digitalRead()`, or I²C/SPI/UART
Examples	LDR, potentiometer, thermistor	Button, PIR motion sensor, DHT22, BME280

L7. Glossary

Term	**Meaning**
Actuator	Converts a control signal and energy into a physical effect (motion, sound, heat)
ADC	Analog-to-digital converter: turns a voltage into a number
Anode / cathode	The + and − terminals of a diode or LED
Breadboard	Board with connected strips of holes for building circuits without soldering
BOM	Bill of Materials: list of parts, quantities and specifications
Bit / byte	One binary digit (0 or 1) / a group of 8 bits
Continuity	Whether two points are electrically connected
CPU	Central processing unit: follows program instructions
Datasheet	Manufacturer's technical document for a component
Debouncing	Ignoring the rapid on/off flickers when a mechanical switch changes
Duty cycle	Percentage of time a PWM signal is ON
Floating input	An input pin connected to nothing, so it reads random values
Flyback diode	Diode across an inductive load that absorbs the switch-off voltage spike
GND	Ground: the 0 V reference point
GPIO	General-purpose input/output pin
Hysteresis	Using two thresholds (on/off) so an output does not flicker
LDR	Light-dependent resistor: resistance falls as light increases

Term	Meaning
Logic level	The voltage a board uses for HIGH (5 V on the Uno R3, 3.3 V on many others)
Microcontroller	CPU, memory and I/O pins on one chip
Ohm's law	$V = IR$
Pull-up / pull-down	Resistor that holds an input HIGH / LOW when nothing else drives it
PWM	Pulse-width modulation: fast on/off switching to control average power
RAM	Volatile working memory; lost when power is off
Resolution	Smallest change a measurement system can show
Schematic	Diagram showing how components are connected
Sensor	Measures a physical quantity and gives a usable signal
Serial Monitor	Arduino IDE window that shows text sent by `Serial.print()`
Storage	Non-volatile memory; kept when power is off
Tolerance	How far a part's real value may differ from its labelled value
Voltage divider	Two resistors in series; the output is the voltage at the junction

L8. Practice questions

Question H — Breadboard. A learner puts both legs of the 330 Ω resistor into holes a5 and c5. The LED never lights. Why?

Question I — Multimeter. You want to know how much current the LED draws. Where do you connect the multimeter, and what must you do with the red lead afterwards?

Question J — Resistor bands. A resistor is banded yellow, violet, red, gold. What is its value and tolerance?

Question K — Uno R4 LED resistor. The Uno R4 pin is rated for about 8 mA. With a 5 V output and a 2 V LED, what is the smallest resistor that keeps the current at or below 8 mA? Which E12 value would you use?

Question L — Button. A button is wired from pin 2 to GND, with `pinMode(2, INPUT)` (not `INPUT_PULLUP`). The Serial Monitor shows random 0s and 1s when nobody touches it. Explain why, and give the one-word fix in the code.

Answers

Answer H. a5 and c5 are in the same connected strip (column 5, rows a–e), so the resistor is shorted out and is not in the path between the pin and the LED. Move one leg to a different column.

Answer I. In **series**: break the circuit (for example between the resistor and the LED) and connect the meter across the gap, with the red lead in the mA socket. Afterwards, move the red lead back to the V/Ω socket. Expect about 9 mA (B8).

Answer J. Yellow (4), violet (7), red (×100) → $47 \times 100 = 4700\ \Omega = 4.7\ \text{k}\Omega$, gold = ±5 %.

Answer K.

$$R = \frac{5 - 2}{0.008} = 375\ \Omega$$

The next E12 value above is **390 Ω** ($I = 3/390 \approx 7.7\ \text{mA}$). B5 recommends 470 Ω–1 kΩ to leave a margin, which is also correct.

Answer L. With the button released, the pin is floating: connected to nothing, so it picks up noise. Fix: `INPUT_PULLUP` (then pressed reads LOW, released reads HIGH).

References (additional to hardware.md)

1. Arduino. "Software" — Arduino IDE download. https://www.arduino.cc/en/software
2. Arduino. `pinMode()` reference (INPUT_PULLUP). https://docs.arduino.cc/language-reference/en/functions/digital-io/pinMode/
3. Arduino. "Debounce" example. https://docs.arduino.cc/built-in-examples/digital/Debounce/
4. Arduino. "Digital Pins" — pull-up resistors. https://docs.arduino.cc/learn/microcontrollers/digital-pins/
5. SparkFun Learn. "How to Use a Breadboard." https://learn.sparkfun.com/tutorials/how-to-use-a-breadboard/all
6. SparkFun Learn. "How to Use a Multimeter." https://learn.sparkfun.com/tutorials/how-to-use-a-multimeter/all
7. SparkFun Learn. "Resistors" (colour codes, standard values). https://learn.sparkfun.com/tutorials/resistors/all
8. SparkFun Learn. "Pull-up Resistors." https://learn.sparkfun.com/tutorials/pull-up-resistors/all
9. SparkFun Learn. "How to Read a Schematic." https://learn.sparkfun.com/tutorials/how-to-read-a-schematic/all
10. Wokwi — online Arduino simulator. https://wokwi.com/

Status note

Drafted 23 Oct 2026 with AI assistance (Claude). The team still needs to check every value against the cited sources, try the activities in a lab session, and record this AI use in the project log before marking any section *Checked*.